

1. Objects and Classes

Author -- Yucheng Wu

Version -- 26.1.25

Example 1.1

```
public class Student {
    // Fields
    private String name;
    private int age;

    // Constructor (initialize the object)
    public Student(String name, int age) {
        this.name = name;
        this.age = age;
    }

    // Methods
    public void introduce() {
        // Print out self introduction
        System.out.println("Hi, I'm " + name + ", age " + age);
    }
}
```

1.1. Object

Definition

An object is an instance of a class; it represents a real-world entity in a program.

Features

- Has **state** (stored in fields). State refer to the parameters such as `int`, `boolean`, `char`, `String` ...
- Has **behavior** (defined by methods).

Example

- Classes: `Student`, `Book`, `Library`
- Objects: `Alice` (`Student`), `"Object First with Java"` (`Book`)

Remark

- In example 1.1, a "student" is an object of the class "class".
-

1.2. Class

Definition

A class is a “blueprint or template” for creating objects; it defines what states and behaviors its objects will have.

Template

```
Public class ClassName {  
    // field  
    // constructor  
    // method  
}
```

Remark

- In example 1.1, a "student" is the object of this class, which represented by two variable "name" and "age".
-

1.3. Fields

Definition

Variables that belong to a class; they store the *state of an object*.

Example

- In example 1.1, the field is following

```
private String name;  
private int age;
```

1.4. Constructor

Definition: Special method to initialize new objects.

Features

- Same name as the class.
- No return type (not even `void`).

Example

- In example 1.1, the constructor is

```
public Student(String name, int age) {  
    this.name = name;  
    this.age = age;  
}
```

Remark

- Every class has at least one constructor. If not explicitly defined, Java provides a **default no-argument constructor**.
-

1.5. Methods

Definition

Define the **behavior** of an object; what it can do.

Types of Methods

- **Accessor (getter)**: Returns information without modifying object state.
- **Mutator (setter)**: Modifies object state.

```
public String getName() {  
    return name;  
} // Accessor  
  
public void setAge(int newAge) {  
    age = newAge;  
} // Mutator
```

Example

- In example 1.1, the method is

```
// Methods  
public void introduce() {  
    // Print out self introduction  
    System.out.println("Hi, I'm " + name + ", age " + age);  
}
```

Remark

- Classes usually contain **fields, constructors, and methods**.